MATH 482: Probabilistic Approach to Machine Learning

Term Project 1: Latent Space Manipulation with StyleGAN2

Group Members:

Toprak Özdemir

Hüseyin Çiyik

Fall 2025

Contents

1 Introduction **2**

2 Setup & Methodology **2**

2.1 Environment Configuration 2

2.2 Attribute Classifier Training 2

2.3 Latent Space Exploration 2

2.3.1 Dataset Generation 2

2.3.2 Direction Discovery 3

2.4 Manipulation and Experiments 3

3 Results **3**

3.1 Latent Space Analysis (Question 1) 4

3.1.1 Theoretical Difference: $\mathcal{Z}$ vs. $\mathcal{W}$ 4

3.1.2 Experimental Observation 4

3.2 Method Comparison (Question 2) 4

3.2.1 Cosine Similarity 4

3.2.2 Visual Quality Comparison 5

3.3 Entanglement Analysis (Question 3) 5

3.4 Failure Cases (Question 4) 6

3.4.1 Alpha Scaling Limits 6

3.4.2 Semantic Failure (Pose) 7

4 Conclusion **7**

A Code Snippets **8**

A.1 Classifier Training and Evaluation 8

A.2 Latent Dataset Generation and Labeling 9

A.3 Direction Discovery 11

A.4 Manipulation Function and Seed Image 11

A.5 Experiments 12

A.5.1 Z-Space vs. W-Space 12

A.5.2 SVM vs. Mean Difference 12

A.5.3 Alpha Scaling 13

1 Introduction

In this project, we explore the latent space of StyleGAN2 trained on the FFHQ dataset. Our objective is to identify semantic directions corresponding to specific facial attributes (specifically "Smiling") and manipulate these attributes using linear interpolation in latent space. We compare the properties of the initial latent space $\mathcal{Z}$ versus the intermediate disentangled space $\mathcal{W}$, and evaluate two different methods for direction discovery: Mean Difference and Linear SVM.

2 Setup & Methodology

2.1 Environment Configuration

The project was implemented in a Python 3.11 environment with GPU acceleration (Tesla T4). We utilized the official **StyleGAN2-ADA PyTorch** repository for the generative model. We loaded the pre-trained **FFHQ** (Flickr-Faces-HQ) network weights (`ffhq.pkl`) to generate high-resolution human faces.

2.2 Attribute Classifier Training

To manipulate specific attributes, we first required a discriminative model to label the generated images.

- **Dataset:** We utilized the **CelebA** dataset, which contains over 200,000 celebrity images with 40 attribute annotations.
- **Architecture:** We employed a pretrained **ResNet-18** model. The final fully connected layer was modified to output a single logit for binary classification.
- **Training:** The model was fine-tuned to detect the *Smiling* attribute (Index 31). We used `BCEWithLogitsLoss` and the Adam optimizer ($lr = 0.001$). Epoch number is chosen to be 2 because after the first epoch loss function started to fluctuate around 0.170 and the final loss is calculated as 0.165.
- **Evaluation:** The trained model has been used on the validation set (20% of the sample) to calculate the accuracy.

The classifier achieved a validation accuracy of approximately 92.5%, which was sufficient for labeling the latent vectors in subsequent steps. The training and evaluation implementation is detailed in **Appendix A.1**. The seed is chosen to be `168` for reproducibility.

2.3 Latent Space Exploration

2.3.1 Dataset Generation

To discover the semantic direction corresponding to "Smiling," we constructed a labeled dataset of latent vectors.

1. **Sampling:** We sampled 5,000 random latent vectors $z \in \mathcal{Z}$ from a standard normal distribution. The seed is still `168`.

2. **Mapping:** These vectors were passed through the StyleGAN2 mapping network to obtain the corresponding intermediate vectors $w \in \mathcal{W}$.

3. **Synthesis & Labeling:** We generated the corresponding images $G(w)$, resized them to 224×224, and passed them through our trained ResNet classifier. Images with a classification probability $p > 0.5$ were labeled as positive (1), and others as negative (0).

This process resulted in a paired dataset $(\mathbf{z}, \mathbf{w}, y)$ of 5,000 samples, which served as the training data for our direction discovery methods (SVM and Mean Difference). The implementation of this, in the code, can be found in **Appendix A.2**.

2.3.2 Direction Discovery

We calculated the semantic direction n using two methods (see **Appendix A.3**):

1. **Mean Difference:** $n_{mean} = \mu_{pos} - \mu_{neg}$

2. **Linear SVM:** n_{svm} is the normal vector to the separating hyperplane.

2.4 Manipulation and Experiments

All manipulations follow the linear interpolation formula:

$$v_{new} = v_{old} + \alpha \cdot n$$

Before the experiments, we created a specific $z \in \mathcal{Z}$ vector and the corresponding $w \in \mathcal{W}$ vector for appropriate comparisons. The seed is chosen to be `198` for reproducibility. The implementation of these can be found in **Appendix A.4**.

Using this methodology, we ran three experiments:

1. **$\mathcal{Z}$-Space vs. $\mathcal{W}$-Space:** To compare them properly, we fixed the direction finding method (SVM) and alpha ($\alpha = 2.0$). Then, we manipulated the seed image both in $\mathcal{Z}$-Space and $\mathcal{W}$-Space.

2. **SVM vs. Mean Difference:** We manipulated the seed image in $\mathcal{W}$-space by using the both methods for comparison.

3. **Alpha Scaling:** We manipulated the seed image in $\mathcal{W}$-space using the SVM direction method with different alpha values for better comparison. The alpha values we used in the experiment are $-7, -5, -2, 0, 2, 5, 7$.

The implementations for the experiments can be found in **Appendix A.5**.

3 Results

Based on this methodology and experiments, we tried to understand differences of latent spaces, direction finding methods and alpha values. Moreover, we investigated the cases where the model fails, e.g. extreme alpha values.

3.1 Latent Space Analysis (Question 1)

Before the experimental observations, the theoretical explanations of latent space differences should be explained.

3.1.1 Theoretical Difference: $\mathcal{Z}$ vs. $\mathcal{W}$

The core innovation of StyleGAN is the mapping network.

- **$\mathcal{Z}$-Space:** The input latent code $z \in \mathcal{Z}$ is sampled from a standard normal distribution $\mathcal{N}(0, I)$. This space is "entangled" because the generator must "bend" the input to match the probability density of the training data. For example, if "long hair" and "feminine features" are correlated in the dataset, the structure of $\mathcal{Z}$ forces these features to be entangled in the representation.
- **$\mathcal{W}$-Space:** The mapping network $f : \mathcal{Z} \to \mathcal{W}$ transforms z into an intermediate vector w. This space is not restricted to a fixed probability distribution, allowing it to "unbend" the data manifold. This results in a "disentangled" representation where linear directions correspond more consistently to single semantic changes.

3.1.2 Experimental Observation

Our experiments confirm this theory. As seen in Figure 1, manipulating the vector in $\mathcal{Z}$-space resulted in significant changes to the subject's identity and adding glasses, whereas $\mathcal{W}$-space manipulation preserved the identity while only altering the target attribute.



(a) Original (seed = 198) (b) $\mathcal{Z}$-Space Manipulation (c) $\mathcal{W}$-Space Manipulation

Figure 1: Comparison of manipulation in $\mathcal{Z}$ vs $\mathcal{W}$ space ($\alpha = 2.0$ with SVM). Note how W-space preserves identity better.

3.2 Method Comparison (Question 2)

3.2.1 Cosine Similarity

We computed the cosine similarity between the two direction vectors in $\mathcal{W}$-space:

$$\text{Cosine Similarity} \approx 0.3315$$

This value corresponds to an angle of approximately 70°. This indicates that while both methods generally point towards the "Smiling" region, they take significantly different paths.

3.2.2 Visual Quality Comparison

The **Linear SVM** yielded better perceptual results.

- The **Mean Difference** vector captures the average difference between populations. Since the dataset contains biases (e.g., smiling faces might be correlated with specific lighting or gender or artifacts), the Mean Difference vector entangles these biases into the direction. In this case, Mean Difference method adds glasses to the subject and changes the shape of the face more.
- The **SVM** finds the decision boundary that best separates the classes. This discriminative approach isolates the specific feature change required to flip the label, resulting in a more clear transformation with fewer artifacts.



(a) Mean Difference Method



(b) SVM Method

Figure 2: Visual comparison of direction finding methods (in $\mathcal{W}$-space with $\alpha = 2.0$).

3.3 Entanglement Analysis (Question 3)

Entanglement occurs when changing one attribute inadvertently changes another. We observed this frequently in $\mathcal{Z}$-space and/or while using the Mean Difference method. Apart from the main seed (198), that we can see $\mathcal{Z}$-space manipulation adds glasses, there are more examples of entanglement.

In Figure 3, we demonstrate a clear case of dataset bias entanglement. When applying the "Smiling" vector, found by the SVM method, to a male subject, the model inadvertently changes the gender and the shirt color. This reveals that the semantic concept of "Smiling" in the training data is statistically correlated with gender. Note that $\alpha = 2.0$



(a) Original (seed = 301)



(b) Z-space Manipulation with SVM

Figure 3: Example of Entanglement: Applying the "Smiling" vector also alters gender/-makeup or background lighting.

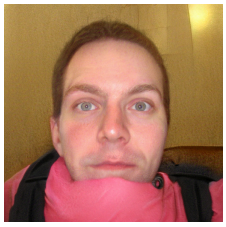
In addition, we observed more entanglement scenarios with different seed images, though we used the SVM method for all of them. All these examples, in Figure 4, of the $\mathcal{Z}$-space manipulations demonstrate the entanglement of attributes.



(a) Original (seed = 151)



(b) Z-space Manipulation with SVM



(c) Original (seed = 1596)



(d) Z-space Manipulation with SVM

Figure 4: From (a) to (b), the manipulation makes the child older ($\alpha = 2.0$).
From (c) to (d), the manipulation distorts the image and change the gender ($\alpha = 3.0$)

3.4 Failure Cases (Question 4)

We explored the limits of the linear model by identifying cases where manipulation fails.

3.4.1 Alpha Scaling Limits

Linearity in the latent space only holds locally. As shown in Figure 5, applying extreme values (e.g., $\alpha = 5.0$ or 7.0) pushes the latent vector into low-density regions of the distribution, resulting in severe artifacts and realistic distortions.



(a) $\alpha = 0$



(b) $\alpha = 2$



(c) $\alpha = 5$



(d) $\alpha = 7$



(e) $\alpha = -2$



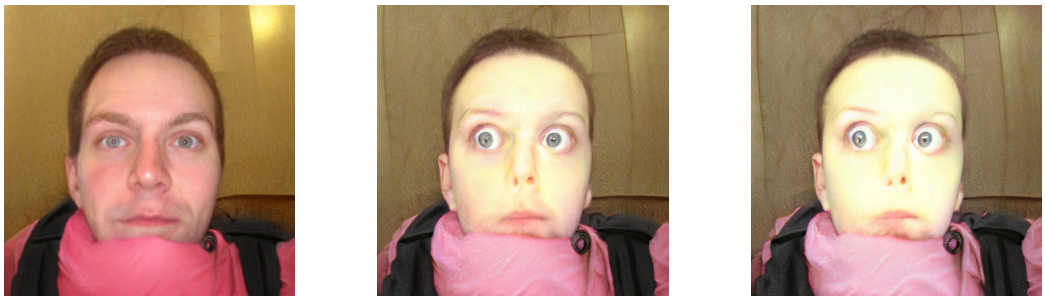
(f) $\alpha = -5$



(g) $\alpha = -7$

Figure 5: Comparison of different alpha values (in $\mathcal{W}$-space with SVM method)

Additionally, negative extreme alpha values distort the image as well in many cases. Figure 6 shows how the manipulation makes the original image unrealistic.

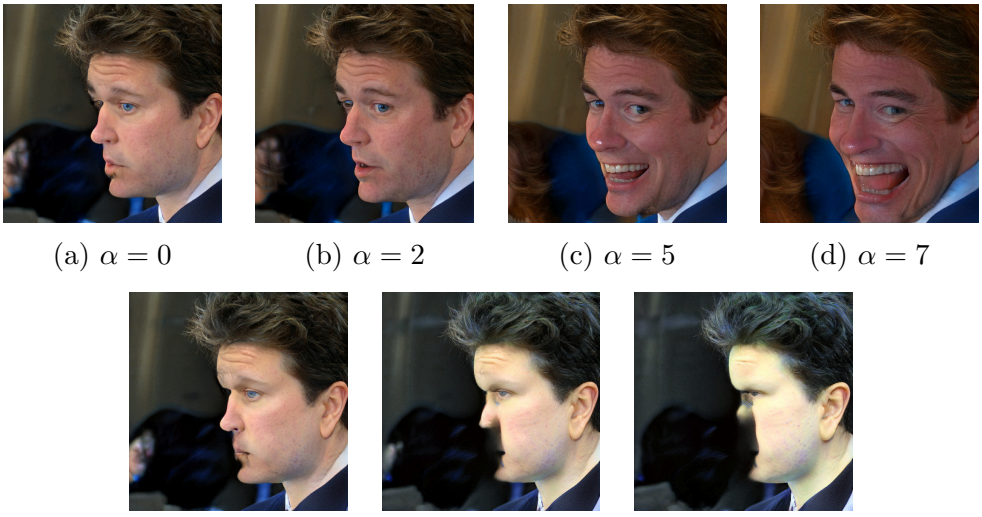


(a) Original (seed = 1596) (b) $\alpha = -5.0$ (c) $\alpha = -7.0$

Figure 6: Result of negative extreme alpha values in $\mathcal{W}$-space with SVM direction vector.

3.4.2 Semantic Failure (Pose)

Figure 7 illustrates a failure case involving a profile-view face. The StyleGAN2 generator struggles to rotate the facial geometry to accommodate a smile, resulting in distorted jawlines and background. This highlights that the linear direction n cannot account for complex 3D geometric constraints. Moderate alpha values cannot manipulate properly, while extreme alpha values distorts the image. The seed is `3879`.



(a) $\alpha = 0$ (b) $\alpha = 2$ (c) $\alpha = 5$ (d) $\alpha = 7$

(e) $\alpha = -2$ (f) $\alpha = -5$ (g) $\alpha = -7$

Figure 7: Failure Case: Manipulation breaks the geometry of a profile-view face.

4 Conclusion

In this project, we successfully manipulated the latent space of StyleGAN2. We demonstrated that $\mathcal{W}$-space offers superior disentanglement compared to $\mathcal{Z}$-space, and that discriminative methods (SVM) generally identify more precise semantic directions than generative mean statistics. The low cosine similarity (0.3315) between the methods confirms that the definition of a semantic attribute is high-dimensional and complex.

A Code Snippets

A.1 Classifier Training and Evaluation

```
    # Section 4 of the Notebook
import torchvision.transforms as transforms
import torchvision.datasets as datasets
from torch.utils.data import DataLoader, random_split, Dataset
from PIL import Image
import pandas as pd
import random

#Reproducibility
SEED = 168
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

# Split dataset (CelebA)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(
    dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(SEED)
)

# Dataloaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True,
    num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False,
    num_workers=2)

print(f"Training samples: {len(train_dataset)}")
print(f"Validation samples: {len(val_dataset)}")

    # Section 5 of the Notebook
import torch.nn as nn
import torch.optim as optim
import torchvision.models as models

# Attribute Index for 'Smiling' in CelebA is 31
ATTR_INDEX = 31

# Load ResNet18
classifier = models.resnet18(weights='DEFAULT')

# Modify final layer for binary classification (1 output logit)
num_ftrs = classifier.fc.in_features
classifier.fc = nn.Linear(num_ftrs, 1)

classifier = classifier.to(device)

# Loss and Optimizer
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(classifier.parameters(), lr=0.001)

```

```
# Training Loop
num_epochs = 2
classifier.train()

for epoch in range(num_epochs):
    running_loss = 0.0
    for i, (inputs, targets) in enumerate(train_loader):
        inputs = inputs.to(device)

        # Extract specific attribute and convert to float
        labels = targets[:, ATTR_INDEX].float().unsqueeze(1).to(device)

        optimizer.zero_grad()
        outputs = classifier(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        if i % 100 == 99:
            print(f"[Epoch {epoch+1}, Batch {i+1}] Loss: {running_loss /
    100:.3f}")
            running_loss = 0.0

     # Section 6 of the Notebook
classifier.eval()
correct = 0
total = 0

print("Evaluating on validation set...")

with torch.no_grad():
    for i, (inputs, targets) in enumerate(val_loader):
        inputs = inputs.to(device)
        labels = targets[:, ATTR_INDEX].float().unsqueeze(1).to(device)

        outputs = classifier(inputs)
        predicted = (torch.sigmoid(outputs) > 0.5).float()

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'Validation Accuracy: {accuracy:.2f}%')
```

A.2 Latent Dataset Generation and Labeling

```
    # Section 7 of the Notebook
# --- CONFIGURATION ---
num_samples = 5000
batch_size = 10
# ---------------------
z_space_list = []
w_space_list = []
labels_list = []
# Clear GPU cache before starting massive generation
torch.cuda.empty_cache()
```

```
# Resize transform for StyleGAN output (1024x1024) to Classifier Input (224x224)
resize_normalize = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

print(f"Generating {num_samples} latent vectors and labeling them...")

# Calculate how many iterations we need
num_batches = int(np.ceil(num_samples / batch_size))

with torch.no_grad():
    for i in range(num_batches):
        # Handle the last batch which might be smaller than batch_size
        current_batch_size = min(batch_size, num_samples - (i * batch_size))

        # 1. Generate Z
        z = torch.randn([current_batch_size, G_ema.z_dim]).to(device)

        # 2. Map to W (G.mapping returns w)
        w = G_ema.mapping(z, None)

        # 3. Generate Image
        img_gen = G_ema.synthesis(w, noise_mode='const')

        # 4. Preprocess for classifier: [-1,1] -> [0,1] -> Normalize
        img_gen = (img_gen * 0.5 + 0.5).clamp(0, 1)
        img_batch = resize_normalize(img_gen)

        # 5. Predict Label
        logits = classifier(img_batch)
        probs = torch.sigmoid(logits)
        preds = (probs > 0.5).float().cpu().numpy()

        # 6. Store
        z_space_list.append(z.cpu().numpy())
        w_space_list.append(w[:, 0, :].cpu().numpy()) # Take first w layer
        labels_list.append(preds)

        # Print progress every 50 batches
        if i % 50 == 0:
            print(f"Generated {i * batch_size} / {num_samples} images...")

# Concatenate
Z_data = np.concatenate(z_space_list, axis=0)
W_data = np.concatenate(w_space_list, axis=0)
Labels = np.concatenate(labels_list, axis=0).flatten()

# Ensure we have exactly num_samples (trim if necessary)
Z_data = Z_data[:num_samples]
W_data = W_data[:num_samples]
Labels = Labels[:num_samples]
```

A.3 Direction Discovery

```
      # Section 8 of the Notebook (For Mean Difference)

# Separate indices
pos_idx = Labels == 1
neg_idx = Labels == 0

# Z-space Mean Difference
mu_pos_z = np.mean(Z_data[pos_idx], axis=0)
mu_neg_z = np.mean(Z_data[neg_idx], axis=0)
direction_mean_z = mu_pos_z - mu_neg_z
# Normalize
direction_mean_z = direction_mean_z / np.linalg.norm(direction_mean_z)

# W-space Mean Difference
mu_pos_w = np.mean(W_data[pos_idx], axis=0)
mu_neg_w = np.mean(W_data[neg_idx], axis=0)
direction_mean_w = mu_pos_w - mu_neg_w
# Normalize
direction_mean_w = direction_mean_w / np.linalg.norm(direction_mean_w)

      # Section 9 of the Notebook (For SVM)
from sklearn.svm import LinearSVC

# Z-space SVM
svm_z = LinearSVC(max_iter=10000, dual=False) #False because n_sample >
   n_features
svm_z.fit(Z_data, Labels)
direction_svm_z = svm_z.coef_.flatten()
direction_svm_z = direction_svm_z / np.linalg.norm(direction_svm_z)

# W-space SVM
svm_w = LinearSVC(max_iter=10000, dual=False)
svm_w.fit(W_data, Labels)
direction_svm_w = svm_w.coef_.flatten()
direction_svm_w = direction_svm_w / np.linalg.norm(direction_svm_w)

# Cosine Similarity between SVM and Mean Difference vectors in W-space
dot_product = np.dot(direction_mean_w, direction_svm_w)
print(f"Cosine Similarity (W-Space): {dot_product:.4f}")
```

A.4 Manipulation Function and Seed Image

```
      # Section 10 of the Notebook
def manipulate(latent_vector, direction, alpha):
    return latent_vector + alpha * direction

# Setting the specific seed for the experiments
viz_seed = 198
torch.manual_seed(viz_seed)

# 1. Generate the specific Z vector
z_sample = torch.randn([1, G_ema.z_dim]).to(device)
# 2. Generate the corresponding W vector
w_sample_full = G_ema.mapping(z_sample, None)
w_sample = w_sample_full[:, 0, :] # Extract the [1, 512] vector for math
```

A.5 Experiments

A.5.1 Z-Space vs. W-Space

```
    # Section 11 of the Notebook
# Alpha controls the strength of the manipulation
alpha = 2.0

# 1. Z-space manipulation
# Formula: z_new = z_old + alpha * n
dir_z_t = torch.tensor(direction_svm_z).float().to(device)
z_new = manipulate(z_sample, dir_z_t, alpha)
img_z_manip = G_ema(z_new, None, noise_mode='const')

# 2. W-space manipulation
dir_w_t = torch.tensor(direction_svm_w).float().to(device)
w_new = manipulate(w_sample, dir_w_t, alpha)

# Note: The generator needs shape [1, 18, 512].
# We repeat our manipulated w vector 18 times.
w_new_full = w_new.unsqueeze(1).repeat(1, G_ema.num_ws, 1)
img_w_manip = G_ema.synthesis(w_new_full, noise_mode='const')

# 3. Original Image (for comparison)
img_orig = G_ema(z_sample, None, noise_mode='const')

# Display Side-by-Side
fig, ax = plt.subplots(1, 3, figsize=(15, 5))

# Helper to process image for display
def process_img(tensor):
    return (tensor[0].permute(1,2,0).cpu().detach() * 0.5 + 0.5).clamp
    (0,1)

ax[0].imshow(process_img(img_orig))
ax[0].set_title(f"Original (Seed {viz_seed})")

ax[1].imshow(process_img(img_z_manip))
ax[1].set_title(f"Z-space Manipulation (alpha={alpha})")

ax[2].imshow(process_img(img_w_manip))
ax[2].set_title(f"W-space Manipulation (alpha={alpha})")

for a in ax: a.axis('off')
plt.show()
```

A.5.2 SVM vs. Mean Difference

```
    # Section 12 of the Notebook
# Compare Mean Difference method vs SVM method in W-space
alpha_comp = 2.0

# 1. Apply Mean Difference Direction
dir_mean_w_t = torch.tensor(direction_mean_w).float().to(device)
w_mean_new = manipulate(w_sample, dir_mean_w_t, alpha_comp)
w_mean_full = w_mean_new.unsqueeze(1).repeat(1, G_ema.num_ws, 1)
img_mean = G_ema.synthesis(w_mean_full, noise_mode='const')
```

```
# 2. Apply SVM Direction
dir_svm_w_t = torch.tensor(direction_svm_w).float().to(device)
w_svm_new = manipulate(w_sample, dir_svm_w_t, alpha_comp)
w_svm_full = w_svm_new.unsqueeze(1).repeat(1, G_ema.num_ws, 1)
img_svm = G_ema.synthesis(w_svm_full, noise_mode='const')

# Display 3 images Side-by-Side
fig, ax = plt.subplots(1, 3, figsize=(18, 6))

# Plot Original
ax[0].imshow(process_img(img_orig))
ax[0].set_title(f"Original (Seed {viz_seed})")

# Plot Mean Difference
ax[1].imshow(process_img(img_mean))
ax[1].set_title(f"Mean Diff (alpha={alpha_comp})")

# Plot SVM
ax[2].imshow(process_img(img_svm))
ax[2].set_title(f"SVM Boundary (alpha={alpha_comp})")

for a in ax:
    a.axis('off')
plt.show()
```

A.5.3 Alpha Scaling

```
    # Section 13 of the Notebook
# Explore linearity by applying different alpha values
alphas = [-7, -5, -2, 0, 2, 5, 7]
# We will use the SVM direction in W-space (usually the best result)
dir_svm_w_t = torch.tensor(direction_svm_w).float().to(device)

fig, axes = plt.subplots(1, len(alphas), figsize=(20, 5))

print(f"Generating alpha interpolation for Seed {viz_seed}...")

for i, a in enumerate(alphas):
    # Manipulate
    w_scale = manipulate(w_sample, dir_svm_w_t, a)

    # Synthesis
    w_scale_full = w_scale.unsqueeze(1).repeat(1, G_ema.num_ws, 1)
    img_scale = G_ema.synthesis(w_scale_full, noise_mode='const')

    # Display
    axes[i].imshow((img_scale[0].permute(1,2,0).cpu().detach() * 0.5 +
   0.5).clamp(0,1))
    axes[i].set_title(f"Alpha = {a}")
    axes[i].axis('off')

plt.show()
```